

Problem1

May 3, 2026

0.1 2.1 Cart-pole swing-up with limited actuation

(a)

The convex approximation (LOCP) around the current iterate $(s^{(i)}, u^{(i)})$ is

$$\begin{aligned} \text{minimize}_{s, u} \quad & (s_N - s_{\text{goal}})^\top P (s_N - s_{\text{goal}}) \\ & + \sum_{k=0}^{N-1} [(s_k - s_{\text{goal}})^\top Q (s_k - s_{\text{goal}}) + u_k^\top R u_k] \\ \text{subject to} \quad & s_0 = \bar{s}_0 \\ & u_k \in \mathcal{U}, \quad \forall k \in \{0, \dots, N-1\} \\ & \|u_k - u_k^{(i)}\|_\infty \leq \rho, \quad \forall k \in \{0, \dots, N-1\} \\ & \|s_k - s_k^{(i)}\|_\infty \leq \rho, \quad \forall k \in \{0, \dots, N\} \\ & s_{k+1} = A_k s_k + B_k u_k + c_k, \quad \forall k \in \{0, \dots, N-1\} \end{aligned}$$

where the terminal constraint $s_N = s_{\text{goal}}$ is replaced by the terminal cost $(s_N - s_{\text{goal}})^\top P (s_N - s_{\text{goal}})$, and A_k, B_k, c_k come from the linearization as follow:

$$s_{k+1} = f_{\mathbf{d}}(s_k^{(i)}, u_k^{(i)}) + \frac{\partial f_{\mathbf{d}}}{\partial s}(s_k^{(i)}, u_k^{(i)})(s_k - s_k^{(i)}) + \frac{\partial f_{\mathbf{d}}}{\partial u}(s_k^{(i)}, u_k^{(i)})(u_k - u_k^{(i)}) = A_k s_k + B_k u_k + c_k$$

with

$$A_k = \frac{\partial f_{\mathbf{d}}}{\partial s}(s_k^{(i)}, u_k^{(i)}), \quad B_k = \frac{\partial f_{\mathbf{d}}}{\partial u}(s_k^{(i)}, u_k^{(i)}), \quad c_k = f_{\mathbf{d}}(s_k^{(i)}, u_k^{(i)}) - A_k s_k^{(i)} - B_k u_k^{(i)}$$

(b) Please see `cartpole_swingup_constrained.py`.

(c)

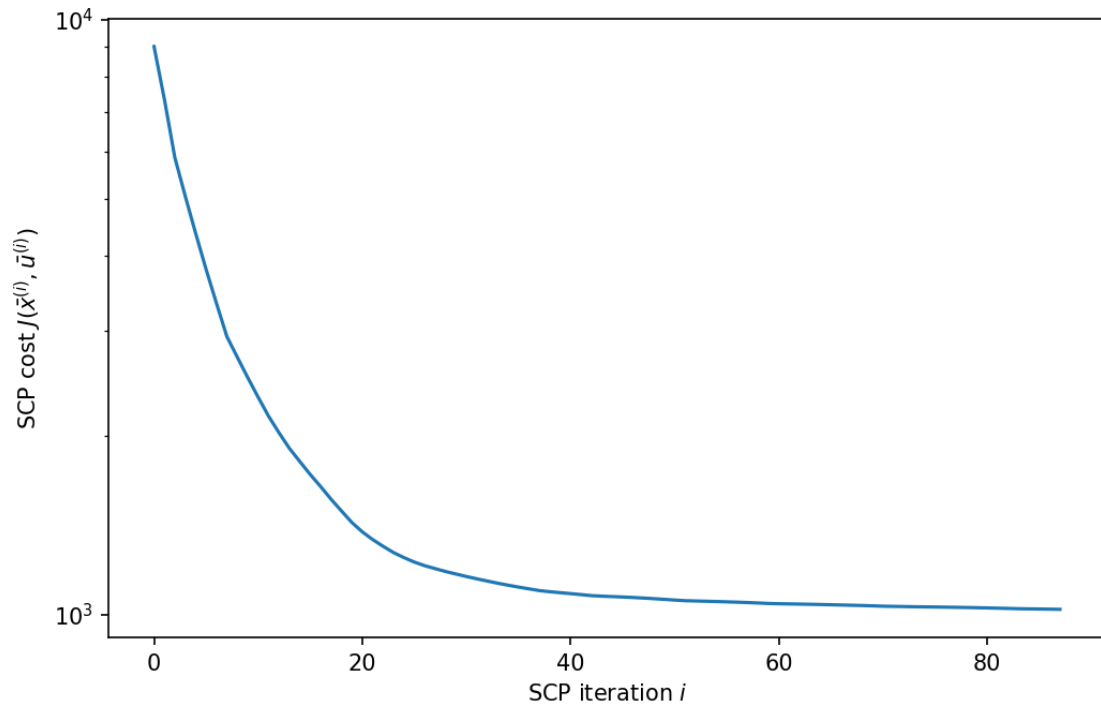
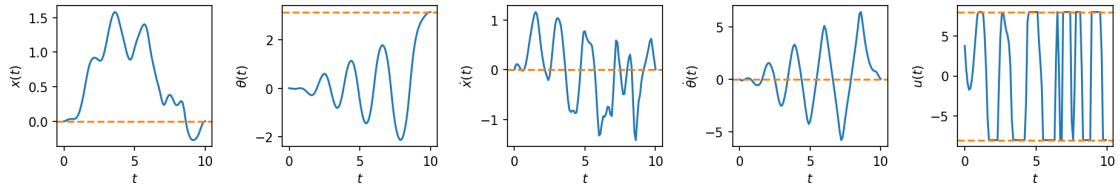
Please see `cartpole_swingup_constrained.py`.

(d)

```
[1]: %run cartpole_swingup_constrained.py
```

```
88%|          | 88/100 [01:21<00:11, 1.08it/s, objective change=0.32265]
```

SCP converged after 88 iterations.



The pendulum just reaches s_{goal} at the end of the horizon but does not stay there. A natural scheme is to use SCP to generate the swing-up trajectory, then switch to a closed-loop LQR controller linearized around the upright equilibrium to keep the pendulum there indefinitely.

Problem2

May 3, 2026

0.1 2.2 Shortest path through a grid

(a)

Doing DP backward (B to A) we compute at each node the optimal cost-to-go (J^* in the figure) and store the optimal edge to follow, represented as a blue arrow. Finally, the optimal path from A to B is computed by always choosing the optimal edge and is highlighted in red. It corresponds to: **up, up, down, up, down, down**.

```
[1]: from IPython.display import Image, display
display(Image(filename='/Users/erwinpoussi/Downloads/NBMetadataCache 2.png',
↪width=700))
```

on a grid. Consider the shortest path problem in a grid. A path is a sequence of nodes $x_0, x_1, \dots, x_n$ such that $x_0 = A$ and $x_n = B$. The cost of a path is the sum of the edge weights. The shortest path problem is to find a path from A to B with minimum cost. In this problem, we will use dynamic programming to find the shortest path. Given a two-dimensional state space, we will use dynamic programming to find the shortest path from each node in the grid.

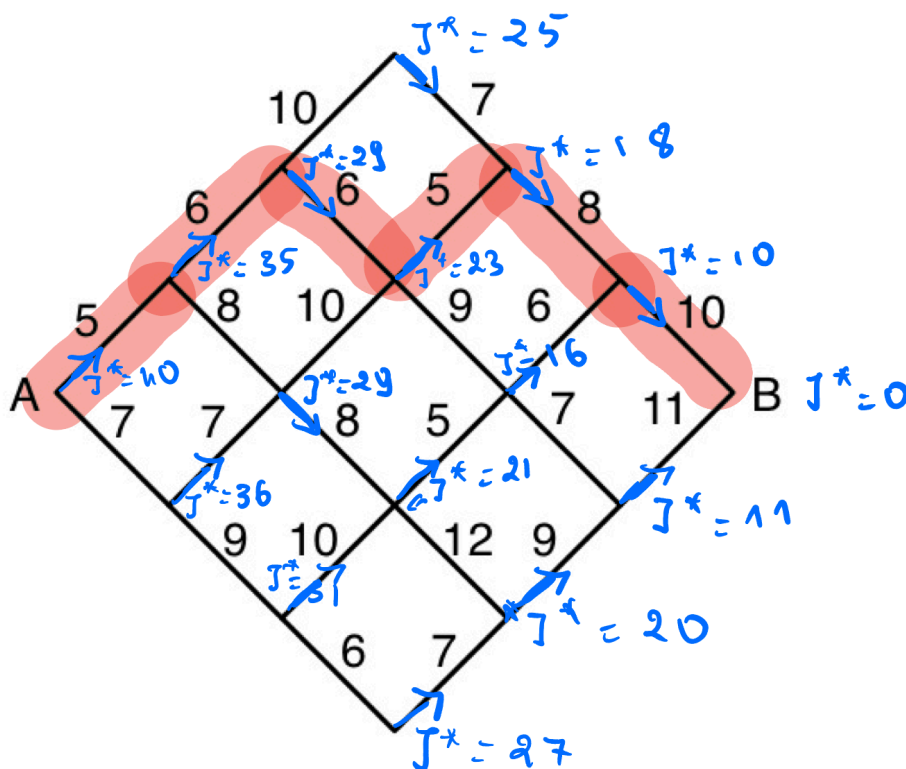


Figure 1: Shortest path problem on a grid.

Dynamic Programming (DP) to find the shortest path from A to B . The grid is a diamond shape. The cost of a path is the sum of the edge weights. The shortest path problem is to find a path from A to B with minimum cost. In this problem, we will use dynamic programming to find the shortest path. Given a two-dimensional state space, we will use dynamic programming to find the shortest path from each node in the grid.

(b)

Number of nodes. For a grid with n segments per side, the diamond has columns $0, 1, \dots, 2n$,

with the number of nodes per column growing from 1 to $n + 1$ then shrinking back to 1:

$$n_{\text{nodes}} = \sum_{k=0}^n (k+1) + \sum_{k=1}^n k = \frac{(n+1)(n+2)}{2} + \frac{n(n+1)}{2} = (n+1)^2.$$

For $n = 3$: $n_{\text{nodes}} = 16$.

Exhaustive search. Every path from A to B consists of $2n$ steps, each either up or down, and must end at B (same row as A), which requires exactly n ups and n downs. The number of such paths is therefore

$$N_{\text{exhaustive}} = \binom{2n}{n} = \frac{(2n)!}{(n!)^2}.$$

For $n = 3$: $\binom{6}{3} = 20$ routes.

Dynamic programming. The DP backward sweep evaluates the cost-to-go J^* once at every node except the terminal B . At each node the computation is a single min over at most 2 successors, so the total number of DP evaluations equals the number of non-terminal nodes:

$$N_{\text{DP}} = (n+1)^2 - 1 = n^2 + 2n.$$

For $n = 3$: $N_{\text{DP}} = 9 + 6 = 15$ evaluations.

Comparison. $\binom{2n}{n}$ grows exponentially ($\sim 4^n / \sqrt{\pi n}$) while $n^2 + 2n$ grows only polynomially, illustrating the considerable computational advantage of DP over exhaustive search.

problem3

May 8, 2026

0.1 2.3 Cart-pole balance

(a)

Starting from the continuous-time dynamics $\dot{s} = f(s, u)$, we apply **Euler integration** to get the discrete-time approximation:

$$s_{k+1} \approx s_k + \Delta t f(s_k, u_k).$$

We then **linearize** f around the equilibrium $(\bar{s}, \bar{u})$ via a first-order Taylor expansion:

$$f(s_k, u_k) \approx f(\bar{s}, \bar{u}) + \left. \frac{\partial f}{\partial s} \right|_{(\bar{s}, \bar{u})} (s_k - \bar{s}) + \left. \frac{\partial f}{\partial u} \right|_{(\bar{s}, \bar{u})} (u_k - \bar{u}).$$

Since $(\bar{s}, \bar{u})$ is an equilibrium, $f(\bar{s}, \bar{u}) = 0$, and $\bar{u} = 0$. Defining $\tilde{s}_k = s_k - \bar{s}$ and subtracting $\bar{s}$ from both sides:

$$\tilde{s}_{k+1} \approx \tilde{s}_k + \Delta t \left(\left. \frac{\partial f}{\partial s} \right|_{(\bar{s}, \bar{u})} \tilde{s}_k + \left. \frac{\partial f}{\partial u} \right|_{(\bar{s}, \bar{u})} u_k \right) = \underbrace{\left(I + \Delta t \left. \frac{\partial f}{\partial s} \right|_{(\bar{s}, \bar{u})} \right)}_A \tilde{s}_k + \underbrace{\Delta t \left. \frac{\partial f}{\partial u} \right|_{(\bar{s}, \bar{u})}}_B u_k.$$

Substituting the given Jacobians evaluated at $\bar{s} = (0, \pi, 0, 0)$, $\bar{u} = 0$:

$$A = I_4 + \Delta t \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p g}{m_c} & 0 & 0 \\ 0 & \frac{(m_c + m_p)g}{m_c \ell} & 0 & 0 \end{bmatrix}, \quad B = \Delta t \begin{bmatrix} 0 \\ 0 \\ \frac{1}{m_c} \\ \frac{1}{m_c \ell} \end{bmatrix}.$$

(b)

```
[1]: import numpy as np
from cartpole_balance import compute_lti_matrices, ricatti_recursion

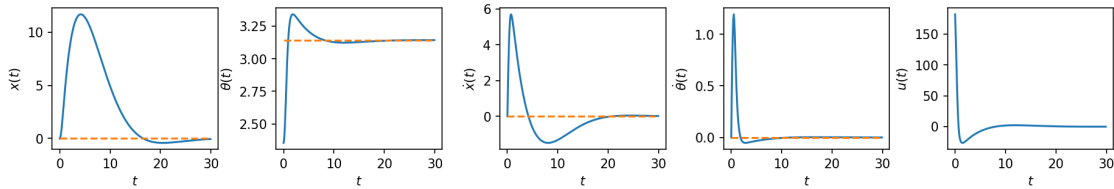
A, B = compute_lti_matrices()
Q, R = np.eye(4), np.eye(1)
K = ricatti_recursion(A, B, Q, R)
print("K_inf:\n", np.round(K, 2))
```

```
K: [[ 0.7291397 -231.85419273  4.21967187 -68.24742822]]
K_inf:
[[ 0.73 -231.85  4.22 -68.25]]
```

(c)

```
[2]: from cartpole_balance import simulate, plot_state_and_control_history

t = np.arange(0.0, 30.0, 0.1)
s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
u_ref = np.array([0.0])
s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
s, u = simulate(t, s_ref, u_ref, s0, K)
plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")
```



```
[3]: from animations import animate_cartpole
from IPython.display import HTML
import matplotlib.pyplot as plt

fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
plt.close(fig)
HTML(ani.to_jshtml())
```

```
[3]: <IPython.core.display.HTML object>
```

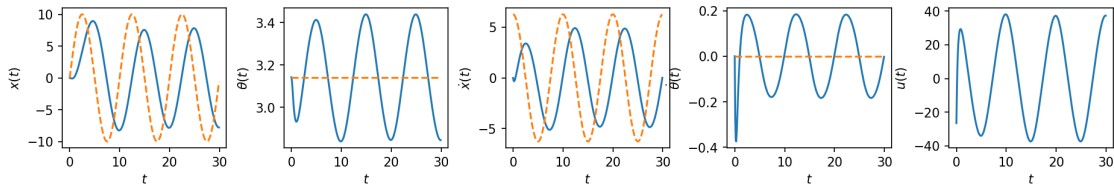
(d)

(i) The Jacobians $\frac{\partial f}{\partial s}$ and $\frac{\partial f}{\partial u}$ do not depend on the cart position x — only on θ , $\dot{\theta}$, and u . Since the reference trajectory always maintains $\bar{\theta}(t) = \pi$, $\dot{\bar{\theta}}(t) = 0$, and $\bar{u} = 0$, the linearization point is identical at every time step regardless of the time-varying $\bar{x}(t)$. Therefore A and B are constant and do not need to be recomputed.

(iii) The desired trajectory $\bar{x}(t) = a \sin(2\pi t/T)$ requires the cart to accelerate ($\ddot{x} \neq 0$), which physically means a nonzero reference force $\bar{u}(t) \neq 0$ is needed to drive it. The LQR was designed assuming $\bar{u} = 0$, so there is no feedforward term to generate this force. As a result the controller cannot simultaneously track $\bar{x}(t)$ and keep $\theta = \pi$: the cart acceleration perturbs the pendulum, causing θ and $\dot{\theta}$ to oscillate. Increasing Q tightens regulation around the reference but does not fix the missing feedforward, so θ and $\dot{\theta}$ keep oscillating.

```
[4]: from cartpole_balance import simulate, reference, plot_state_and_control_history

t = np.arange(0.0, 30.0, 0.1)
s_ref = np.array([reference(ti) for ti in t])
u_ref = np.array([0.0])
s0 = np.array([0.0, np.pi, 0.0, 0.0])
s, u = simulate(t, s_ref, u_ref, s0, K)
plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")
```



```
[5]: from animations import animate_cartpole
from IPython.display import HTML
import matplotlib.pyplot as plt

fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
plt.close(fig)
HTML(ani.to_jshtml())
```

```
[5]: <IPython.core.display.HTML object>
```

```
[ ]:
```


problem4

May 3, 2026

0.1 2.4 Cart-pole swing-up

(a) Please see `cartpole_swingup.py`.

(b)

Using a second order Taylor expansion of the stage cost $c(\tilde{s}_k, \tilde{u}_k)$ around $(\bar{s}_k, \bar{u}_k)$:

$$c(\tilde{s}_k, \tilde{u}_k) = c(\bar{s}_k, \bar{u}_k) + \underbrace{\left(\frac{\partial c}{\partial s} \Big|_{(\bar{s}_k, \bar{u}_k)} \right)}_{=q_k}^\top \tilde{s}_k + \underbrace{\left(\frac{\partial c}{\partial u} \Big|_{(\bar{s}_k, \bar{u}_k)} \right)}_{=r_k}^\top \tilde{u}_k + \underbrace{\frac{1}{2} \tilde{s}_k^\top \frac{\partial^2 c}{\partial s^2} \Big|_{(\bar{s}_k, \bar{u}_k)}}_{=Q} \tilde{s}_k + \underbrace{\frac{1}{2} \tilde{u}_k^\top \frac{\partial^2 c}{\partial u^2} \Big|_{(\bar{s}_k, \bar{u}_k)}}_{=R} \tilde{u}_k$$

hence $\boxed{q_k = Q(\bar{s}_k - s_{\text{goal}})}$ and $\boxed{r_k = R\bar{u}_k}$.

Same for the terminal cost $c_N(\tilde{s}_N)$:

$$c_N(\tilde{s}_N) = c_N(\bar{s}_N) + \underbrace{\left(\frac{\partial c_N}{\partial s} \Big|_{\bar{s}_N} \right)}_{=q_N}^\top \tilde{s}_N + \underbrace{\frac{1}{2} \tilde{s}_N^\top \frac{\partial^2 c_N}{\partial s^2} \Big|_{\bar{s}_N}}_{=Q_N} \tilde{s}_N$$

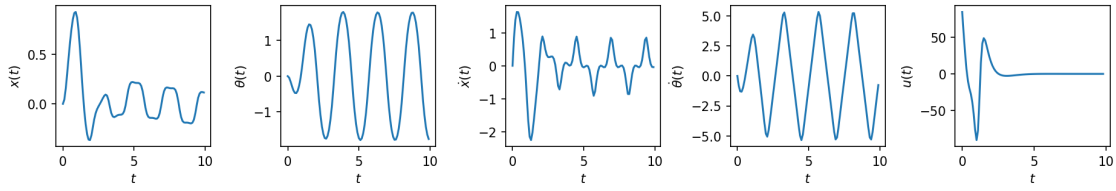
hence $\boxed{q_N = Q_N(\bar{s}_N - s_{\text{goal}})}$.

(c) Please see `cartpole_swingup.py`.

(d) Please see `cartpole_swingup.py`.

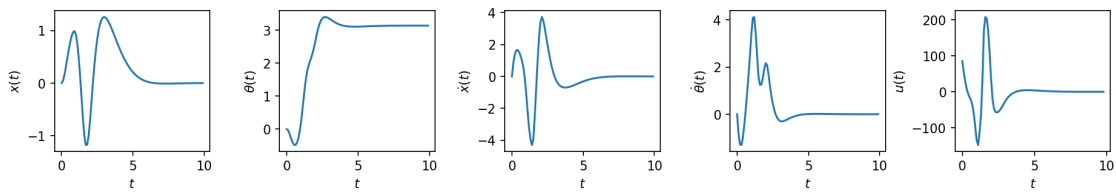
```
[1]: from IPython.display import Image, display
      print("Open-loop:")
      display(Image(filename='cartpole_swingup_ol.png'))
```

Open-loop:



```
[2]: print("Closed-loop:")
      display(Image(filename='cartpole_swingup_cl.png'))
```

Closed-loop:



While the open-loop is consistently oscillating, the closed-loop is progressively damped towards convergence as expected.

Problem5

May 3, 2026

0.1 2.5 Machine maintenance

States: R (running), B (broken) at the start of each week.

Terminal: $J_5(R) = J_5(B) = 0$

Immediate expected net profits and transitions:

- R + Maintain: $0.6(100 - 20) + 0.4(0 - 20) = 40$, next state: R w.p. 0.6, B w.p. 0.4
- R + No maintain: $0.3(100) + 0.7(0) = 30$, next state: R w.p. 0.3, B w.p. 0.7
- B + Repair: $0.6(100 - 40) + 0.4(0 - 40) = 20$, next state: R w.p. 0.6, B w.p. 0.4
- B + Replace: $1(100 - 150) = -50$, next state: R w.p. 1

Week 4:

$J_4(R)$: Maintain $\rightarrow 40$, No maintain $\rightarrow 30 \Rightarrow J_4(R) = \max(40, 30) = \mathbf{40}$ (Maintain)

$J_4(B)$: Repair $\rightarrow 20$, Replace $\rightarrow -50 \Rightarrow J_4(B) = \max(20, -50) = \mathbf{20}$ (Repair)

Week 3:

$J_3(R)$: Maintain $\rightarrow 40 + 0.6(40) + 0.4(20) = 72$, No maintain $\rightarrow 30 + 0.3(40) + 0.7(20) = 56$
 $\Rightarrow J_3(R) = \max(72, 56) = \mathbf{72}$ (Maintain)

$J_3(B)$: Repair $\rightarrow 20 + 0.6(40) + 0.4(20) = 52$, Replace $\rightarrow -50 + 40 = -10 \Rightarrow J_3(B) = \max(52, -10) = \mathbf{52}$ (Repair)

Week 2:

$J_2(R)$: Maintain $\rightarrow 40 + 0.6(72) + 0.4(52) = 104$, No maintain $\rightarrow 30 + 0.3(72) + 0.7(52) = 88$
 $\Rightarrow J_2(R) = \max(104, 88) = \mathbf{104}$ (Maintain)

$J_2(B)$: Repair $\rightarrow 20 + 0.6(72) + 0.4(52) = 84$, Replace $\rightarrow -50 + 72 = 22 \Rightarrow J_2(B) = \max(84, 22) = \mathbf{84}$ (Repair)

Week 1 (new machine, guaranteed to run $\Rightarrow$ earns \$100, ends in state R, no maintain needed):

$$J_1 = 100 + J_2(R) = 100 + 104 = \boxed{204}$$

```

1  """
2  Solution code for the problem "Cart-pole balance".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  import numpy as np
8  from scipy.integrate import odeint
9  import matplotlib.pyplot as plt
10
11 from animations import animate_cartpole
12
13 # Constants
14 n = 4 # state dimension
15 m = 1 # control dimension
16 mp = 2.0 # pendulum mass
17 mc = 10.0 # cart mass
18 L = 1.0 # pendulum length
19 g = 9.81 # gravitational acceleration
20 dt = 0.1 # discretization time step
21 animate = False # whether or not to animate results
22
23
24 def cartpole(s: np.ndarray, u: np.ndarray) -> np.ndarray:
25     """Compute the cart-pole state derivative
26
27     Args:
28         s (np.ndarray): The cartpole state: [x, theta, x_dot, theta_dot], shape (n,)
29         u (np.ndarray): The cartpole control: [F_x], shape (m,)
30
31     Returns:
32         np.ndarray: The state derivative, shape (n,)
33     """
34     x, theta, dx, dtheta = s
35     sintheta, costheta = np.sin(theta), np.cos(theta)
36     h = mc + mp * (sintheta**2)
37     ds = np.array(
38         [
39             dx,
40             dtheta,
41             (mp * sintheta * (L * (dtheta**2) + g * costheta) + u[0]) / h,
42             -((mc + mp) * g * sintheta + mp * L * (dtheta**2) * sintheta * costheta + u[0] * costheta)
43             / (h * L),
44         ]
45     )
46     return ds
47
48
49 def reference(t: float) -> np.ndarray:
50     """Compute the reference state (s_bar) at time t
51
52     Args:
53         t (float): Evaluation time
54
55     Returns:
56         np.ndarray: Reference state, shape (n,)
57     """
58     a = 10.0 # Amplitude
59     T = 10.0 # Period
60
61     # PART (d) #####
62     return np.array([
63         a * np.sin(2 * np.pi * t / T),
64         np.pi,
65         a * (2 * np.pi / T) * np.cos(2 * np.pi * t / T),
66         0.0,
67     ])
68     # END PART (d) #####
69
70
71 def ricatti_recursion(
72     A: np.ndarray, B: np.ndarray, Q: np.ndarray, R: np.ndarray

```

```

73 ) -> np.ndarray:
74     """Compute the gain matrix K through Ricatti recursion
75
76     Args:
77         A (np.ndarray): Dynamics matrix, shape (n, n)
78         B (np.ndarray): Controls matrix, shape (n, m)
79         Q (np.ndarray): State cost matrix, shape (n, n)
80         R (np.ndarray): Control cost matrix, shape (m, m)
81
82     Returns:
83         np.ndarray: Gain matrix K, shape (m, n)
84     """
85     eps = 1e-4 # Riccati recursion convergence tolerance
86     max_iters = 1000 # Riccati recursion maximum number of iterations
87     P_prev = np.zeros((n, n)) # initialization
88     converged = False
89     for i in range(max_iters):
90         # PART (b) #####
91         K = -np.linalg.inv(R + B.T @ P_prev @ B) @ B.T @ P_prev @ A
92         P = Q + A.T @ P_prev @ (A + B @ K)
93         if np.max(np.abs(P - P_prev)) < eps:
94             converged = True
95             break
96         P_prev = P
97     # END PART (b) #####
98     if not converged:
99         raise RuntimeError("Ricatti recursion did not converge!")
100     print("K:", K)
101     return K
102
103
104 def simulate(
105     t: np.ndarray, s_ref: np.ndarray, u_ref: np.ndarray, s0: np.ndarray, K: np.ndarray
106 ) -> tuple[np.ndarray, np.ndarray]:
107     """Simulate the cartpole
108
109     Args:
110         t (np.ndarray): Evaluation times, shape (num_timesteps,)
111         s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
112         u_ref (np.ndarray): Reference control u_bar, shape (m,)
113         s0 (np.ndarray): Initial state, shape (n,)
114         K (np.ndarray): Feedback gain matrix (Ricatti recursion result), shape (m, n)
115
116     Returns:
117         tuple[np.ndarray, np.ndarray]: Tuple of:
118             np.ndarray: The state history, shape (num_timesteps, n)
119             np.ndarray: The control history, shape (num_timesteps, m)
120     """
121
122     def cartpole_wrapper(s, t, u):
123         """Helper function to get cartpole() into a form preferred by odeint, which expects t as the second arg"""
124         return cartpole(s, u)
125
126     # PART (c) #####
127     N = t.size - 1
128     s = np.zeros((t.size, n))
129     u = np.zeros((t.size, m))
130     s[0] = s0
131     for k in range(N):
132         u[k] = K @ (s[k] - s_ref[k]) + u_ref
133         s[k + 1] = odeint(cartpole_wrapper, s[k], t[k : k + 2], (u[k],))[1]
134     u[N] = K @ (s[N] - s_ref[N]) + u_ref
135     # END PART (c) #####
136     return s, u
137
138
139 def compute_lti_matrices() -> tuple[np.ndarray, np.ndarray]:
140     """Compute the linearized dynamics matrices A and B of the LTI system
141
142     Returns:
143         tuple[np.ndarray, np.ndarray]: Tuple of:
144             np.ndarray: The A (dynamics) matrix, shape (n, n)
145             np.ndarray: The B (controls) matrix, shape (n, m)
146     """
147     # PART (a) #####
148     A = np.eye(n) + dt * np.array([

```

```

149     [0, 0, 1, 0],
150     [0, 0, 0, 1],
151     [0, mp * g / mc, 0, 0],
152     [0, (mc + mp) * g / (mc * L), 0, 0],
153 ]
154 B = dt * np.array([[0], [0], [1 / mc], [1 / (mc * L)]])
155 # END PART (a) #####
156 return A, B
157
158
159 def plot_state_and_control_history(
160     s: np.ndarray, u: np.ndarray, t: np.ndarray, s_ref: np.ndarray, name: str
161 ) -> None:
162     """Helper function for cartpole visualization
163
164     Args:
165         s (np.ndarray): State history, shape (num_timesteps, n)
166         u (np.ndarray): Control history, shape (num_timesteps, m)
167         t (np.ndarray): Times, shape (num_timesteps,)
168         s_ref (np.ndarray): Reference state s_bar, evaluated at each time t. Shape (num_timesteps, n)
169         name (str): Filename prefix for saving figures
170
171     """
172     fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
173     plt.subplots_adjust(wspace=0.35)
174     labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
175     labels_u = (r"$u(t)$",)
176     for i in range(n):
177         axes[i].plot(t, s[:, i])
178         axes[i].plot(t, s_ref[:, i], "--")
179         axes[i].set_xlabel(r"$t$")
180         axes[i].set_ylabel(labels_s[i])
181     for i in range(m):
182         axes[n + i].plot(t, u[:, i])
183         axes[n + i].set_xlabel(r"$t$")
184         axes[n + i].set_ylabel(labels_u[i])
185     plt.savefig(f"{name}.png", bbox_inches="tight")
186     plt.show()
187
188     if animate:
189         fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
190         ani.save(f"{name}.mp4", writer="ffmpeg")
191         plt.show()
192
193 def main():
194     # Part A
195     A, B = compute_lti_matrices()
196
197     # Part B
198     Q = np.eye(n) # state cost matrix
199     R = np.eye(m) # control cost matrix
200     K = ricatti_recursion(A, B, Q, R)
201
202     # Part C
203     t = np.arange(0.0, 30.0, 1 / 10)
204     s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
205     u_ref = np.array([0.0])
206     s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
207     s, u = simulate(t, s_ref, u_ref, s0, K)
208     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")
209
210     # Part D
211     # Note: t, u_ref unchanged from part c
212     s_ref = np.array([reference(ti) for ti in t])
213     s0 = np.array([0.0, np.pi, 0.0, 0.0])
214     s, u = simulate(t, s_ref, u_ref, s0, K)
215     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")
216
217
218 if __name__ == "__main__":
219     main()

```

```

1  """
2  Starter code for the problem "Cart-pole swing-up".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  import time
8
9  from animations import animate_cartpole
10
11 import jax
12 import jax.numpy as jnp
13
14 import matplotlib.pyplot as plt
15
16 import numpy as np
17
18 from scipy.integrate import odeint
19
20
21 def linearize(f, s, u):
22     """Linearize the function `f(s, u)` around `(s, u)`.
23
24     Arguments
25     -----
26     f : callable
27         A nonlinear function with call signature `f(s, u)`.
28     s : numpy.ndarray
29         The state (1-D).
30     u : numpy.ndarray
31         The control input (1-D).
32
33     Returns
34     -----
35     A : numpy.ndarray
36         The Jacobian of `f` at `(s, u)`, with respect to `s`.
37     B : numpy.ndarray
38         The Jacobian of `f` at `(s, u)`, with respect to `u`.
39     """
40     # WRITE YOUR CODE BELOW #####
41     # INSTRUCTIONS: Use JAX to compute `A` and `B` in one line.
42     A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
43     #####
44     return A, B
45
46
47 def ilqr(f, s0, s_goal, N, Q, R, QN, eps=1e-3, max_iters=1000):
48     """Compute the iLQR set-point tracking solution.
49
50     Arguments
51     -----
52     f : callable
53         A function describing the discrete-time dynamics, such that
54         `s[k+1] = f(s[k], u[k])`.
55     s0 : numpy.ndarray
56         The initial state (1-D).

```

```

57     s_goal : numpy.ndarray
58         The goal state (1-D).
59     N : int
60         The time horizon of the LQR cost function.
61     Q : numpy.ndarray
62         The state cost matrix (2-D).
63     R : numpy.ndarray
64         The control cost matrix (2-D).
65     QN : numpy.ndarray
66         The terminal state cost matrix (2-D).
67     eps : float, optional
68         Termination threshold for iLQR.
69     max_iters : int, optional
70         Maximum number of iLQR iterations.
71
72     Returns
73     -----
74     s_bar : numpy.ndarray
75         A 2-D array where `s_bar[k]` is the nominal state at time step `k`,
76         for `k = 0, 1, ..., N-1`
77     u_bar : numpy.ndarray
78         A 2-D array where `u_bar[k]` is the nominal control at time step `k`,
79         for `k = 0, 1, ..., N-1`
80     Y : numpy.ndarray
81         A 3-D array where `Y[k]` is the matrix gain term of the iLQR control
82         law at time step `k`, for `k = 0, 1, ..., N-1`
83     y : numpy.ndarray
84         A 2-D array where `y[k]` is the offset term of the iLQR control law
85         at time step `k`, for `k = 0, 1, ..., N-1`
86     """
87     if max_iters <= 1:
88         raise ValueError("Argument `max_iters` must be at least 1.")
89     n = Q.shape[0] # state dimension
90     m = R.shape[0] # control dimension
91
92     # Initialize gains `Y` and offsets `y` for the policy
93     Y = np.zeros((N, m, n))
94     y = np.zeros((N, m))
95
96     # Initialize the nominal trajectory `(s_bar, u_bar)`, and the
97     # deviations `(ds, du)`
98     u_bar = np.zeros((N, m))
99     s_bar = np.zeros((N + 1, n))
100    s_bar[0] = s0
101    for k in range(N):
102        s_bar[k + 1] = f(s_bar[k], u_bar[k])
103    ds = np.zeros((N + 1, n))
104    du = np.zeros((N, m))
105
106    # iLQR loop
107    converged = False
108    for _ in range(max_iters):
109        # Linearize the dynamics at each step `k` of `(s_bar, u_bar)`
110        A, B = jax.vmap(linearize, in_axes=(None, 0, 0))(f, s_bar[:-1], u_bar)
111        A, B = np.array(A), np.array(B)
112
113        # PART (c) #####
114        # INSTRUCTIONS: Update `Y`, `y`, `ds`, `du`, `s_bar`, and `u_bar`.
115
116        # Backward pass: Riccati recursion for P (value Hessian), p (value gradient),

```



```

117     # gains Y[k] and offsets y[k]
118     P = QN
119     p = QN @ (s_bar[-1] - s_goal)
120     for k in range(N - 1, -1, -1):
121         inv = np.linalg.inv(R + B[k].T @ P @ B[k])
122         Y[k] = -inv @ B[k].T @ P @ A[k]
123         y[k] = -inv @ (R @ u_bar[k] + B[k].T @ p)
124         p = Q @ (s_bar[k] - s_goal) + A[k].T @ (p + P @ B[k] @ y[k])
125         P = Q + A[k].T @ P @ (A[k] + B[k] @ Y[k])
126
127     # Forward pass: apply policy on real dynamics
128     s_new = np.zeros_like(s_bar)
129     u_new = np.zeros_like(u_bar)
130     s_new[0] = s0
131     for k in range(N):
132         du[k] = Y[k] @ (s_new[k] - s_bar[k]) + y[k]
133         u_new[k] = u_bar[k] + du[k]
134         s_new[k + 1] = f(s_new[k], u_new[k])
135     s_bar = s_new
136     u_bar = u_new
137
138     #####
139
140     if np.max(np.abs(du)) < eps:
141         converged = True
142         break
143     if not converged:
144         raise RuntimeError("iLQR did not converge!")
145     return s_bar, u_bar, Y, y
146
147
148 def cartpole(s, u):
149     """Compute the cart-pole state derivative."""
150     mp = 2.0 # pendulum mass
151     mc = 10.0 # cart mass
152     L = 1.0 # pendulum length
153     g = 9.81 # gravitational acceleration
154
155     x, theta, dx, dtheta = s
156     sintheta, costheta = jnp.sin(theta), jnp.cos(theta)
157     h = mc + mp * (sintheta**2)
158     ds = jnp.array(
159         [
160             dx,
161             dtheta,
162             (mp * sintheta * (L * (dtheta**2) + g * costheta) + u[0]) / h,
163             -((mc + mp) * g * sintheta + mp * L * (dtheta**2) * sintheta * costheta + u[0] * costheta)
164             / (h * L),
165         ]
166     )
167     return ds
168
169
170 # Define constants
171 n = 4 # state dimension
172 m = 1 # control dimension
173 Q = np.diag(np.array([10.0, 10.0, 2.0, 2.0])) # state cost matrix
174 R = 1e-2 * np.eye(m) # control cost matrix
175 QN = 1e2 * np.eye(n) # terminal state cost matrix
176 s0 = np.array([0.0, 0.0, 0.0, 0.0]) # initial state

```

```

177 s_goal = np.array([0.0, np.pi, 0.0, 0.0]) # goal state
178 T = 10.0 # simulation time
179 dt = 0.1 # sampling time
180 animate = False # flag for animation
181 closed_loop = True # flag for closed-loop control
182
183 # Initialize continuous-time and discretized dynamics
184 f = jax.jit(cartpole)
185 fd = jax.jit(lambda s, u, dt=dt: s + dt * f(s, u))
186
187 # Compute the iLQR solution with the discretized dynamics
188 print("Computing iLQR solution ... ", end="", flush=True)
189 start = time.time()
190 t = np.arange(0.0, T, dt)
191 N = t.size - 1
192 s_bar, u_bar, Y, y = ilqr(fd, s0, s_goal, N, Q, R, QN)
193 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
194
195 # Simulate on the true continuous-time system
196 print("Simulating ... ", end="", flush=True)
197 start = time.time()
198 s = np.zeros((N + 1, n))
199 u = np.zeros((N, m))
200 s[0] = s0
201 for k in range(N):
202     # PART (d) #####
203     # INSTRUCTIONS: Compute either the closed-loop or open-loop value of
204     # `u[k]`, depending on the Boolean flag `closed_loop`.
205     if closed_loop:
206         u[k] = u_bar[k] + Y[k] @ (s[k] - s_bar[k]) + y[k]
207     else: # do open-loop control
208         u[k] = u_bar[k]
209     #####
210     s[k + 1] = odeint(lambda s, t: f(s, u[k]), s[k], t[k : k + 2])[1]
211 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
212
213 # Plot
214 fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
215 plt.subplots_adjust(wspace=0.45)
216 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
217 labels_u = (r"$u(t)$",)
218 for i in range(n):
219     axes[i].plot(t, s[:, i])
220     axes[i].set_xlabel(r"$t$")
221     axes[i].set_ylabel(labels_s[i])
222 for i in range(m):
223     axes[n + i].plot(t[:-1], u[:, i])
224     axes[n + i].set_xlabel(r"$t$")
225     axes[n + i].set_ylabel(labels_u[i])
226 if closed_loop:
227     plt.savefig("cartpole_swingup_cl.png", bbox_inches="tight")
228 else:
229     plt.savefig("cartpole_swingup_ol.png", bbox_inches="tight")
230 plt.show()
231
232 if animate:
233     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
234     ani.save("cartpole_swingup.mp4", writer="ffmpeg")
235     plt.show()

```

```

1  """
2  Starter code for the problem "Cart-pole swing-up with limited actuation".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  from functools import partial
8
9  from animations import animate_cartpole
10
11 import cvxpy as cvx
12
13 import jax
14 import jax.numpy as jnp
15
16 import matplotlib.pyplot as plt
17
18 import numpy as np
19
20 from tqdm import tqdm
21
22
23 @partial(jax.jit, static_argnums=(0,))
24 @partial(jax.vmap, in_axes=(None, 0, 0))
25 def affinize(f, s, u):
26     """Affinize the function `f(s, u)` around `(s, u)`.

```

```

71     The terminal state cost matrix (2-D).
72     Q : numpy.ndarray
73     The state stage cost matrix (2-D).
74     R : numpy.ndarray
75     The control stage cost matrix (2-D).
76     u_max : float
77     The bound defining the control set `[-u_max, u_max]`.
78     ρ : float
79     Trust region radius.
80     eps : float
81     Termination threshold for SCP.
82     max_iters : int
83     Maximum number of SCP iterations.
84
85     Returns
86     -----
87     s : numpy.ndarray
88         A 2-D array where `s[k]` is the open-loop state at time step `k`,
89         for `k = 0, 1, ..., N-1`
90     u : numpy.ndarray
91         A 2-D array where `u[k]` is the open-loop state at time step `k`,
92         for `k = 0, 1, ..., N-1`
93     J : numpy.ndarray
94         A 1-D array where `J[i]` is the SCP sub-problem cost after the i-th
95         iteration, for `i = 0, 1, ..., (iteration when convergence occurred)`
96     """
97     n = Q.shape[0] # state dimension
98     m = R.shape[0] # control dimension
99
100    # Initialize dynamically feasible nominal trajectories
101    u = np.zeros((N, m))
102    s = np.zeros((N + 1, n))
103    s[0] = s0
104    for k in range(N):
105        s[k + 1] = fd(s[k], u[k])
106
107    # Do SCP until convergence or maximum number of iterations is reached
108    converged = False
109    J = np.zeros(max_iters + 1)
110    J[0] = np.inf
111    for i in (prog_bar := tqdm(range(max_iters))):
112        s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max, ρ)
113        dJ = np.abs(J[i + 1] - J[i])
114        prog_bar.set_postfix({"objective change": "{:.5f}".format(dJ)})
115        if dJ < eps:
116            converged = True
117            print("SCP converged after {} iterations.".format(i))
118            break
119    if not converged:
120        raise RuntimeError("SCP did not converge!")
121    J = J[1 : i + 1]
122    return s, u, J
123
124
125 def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max, ρ):
126     """Solve a single SCP sub-problem for the cart-pole swing-up problem.
127
128     Arguments
129     -----
130     f : callable
131         A function describing the discrete-time dynamics, such that
132         `s[k+1] = f(s[k], u[k])`.
133     s0 : numpy.ndarray
134         The initial state (1-D).
135     s_goal : numpy.ndarray
136         The goal state (1-D).
137     s_prev : numpy.ndarray
138         The state trajectory around which the problem is convexified (2-D).
139     u_prev : numpy.ndarray
140         The control trajectory around which the problem is convexified (2-D).
141     N : int
142         The time horizon of the LQR cost function.
143     P : numpy.ndarray
144         The terminal state cost matrix (2-D).

```

```

145 Q : numpy.ndarray
146     The state stage cost matrix (2-D).
147 R : numpy.ndarray
148     The control stage cost matrix (2-D).
149 u_max : float
150     The bound defining the control set `[-u_max, u_max]`.
151 ρ : float
152     Trust region radius.
153
154 Returns
155 -----
156 s : numpy.ndarray
157     A 2-D array where `s[k]` is the open-loop state at time step `k`,
158     for `k = 0, 1, ..., N-1`
159 u : numpy.ndarray
160     A 2-D array where `u[k]` is the open-loop state at time step `k`,
161     for `k = 0, 1, ..., N-1`
162 J : float
163     The SCP sub-problem cost.
164 """
165 A, B, c = affinize(f, s_prev[:-1], u_prev)
166 A, B, c = np.array(A), np.array(B), np.array(c)
167 n = Q.shape[0]
168 m = R.shape[0]
169 s_cvx = cvx.Variable((N + 1, n))
170 u_cvx = cvx.Variable((N, m))
171
172 # PART (c) #####
173 # INSTRUCTIONS: Construct the convex SCP sub-problem.
174 objective = 0.0
175 constraints = []
176 constraints += [s_cvx[0] == s0]
177 constraints += [cvx.norm(s_cvx[k] - s_prev[k], 'inf') <= ρ for k in range(N+1)]
178 constraints += [cvx.norm(u_cvx[k] - u_prev[k], 'inf') <= ρ for k in range(N)]
179 constraints += [cvx.norm(u_cvx[k], 'inf') <= u_max for k in range(N)]
180 constraints += [s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k] for k in range(N)]
181 objective += cvx.quad_form(s_cvx[N] - s_goal, P)
182 objective += cvx.sum([cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.quad_form(u_cvx[k], R) for k in range(N)])
183 # END PART (c) #####
184
185 prob = cvx.Problem(cvx.Minimize(objective), constraints)
186 prob.solve()
187 if prob.status != "optimal":
188     raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
189 s = s_cvx.value
190 u = u_cvx.value
191 J = prob.objective.value
192 return s, u, J
193
194
195 def cartpole(s, u):
196     """Compute the cart-pole state derivative."""
197     mp = 1.0 # pendulum mass
198     mc = 4.0 # cart mass
199     L = 1.0 # pendulum length
200     g = 9.81 # gravitational acceleration
201
202     x, θ, dx, dθ = s
203     sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
204     h = mc + mp * (sinθ**2)
205     ds = jnp.array(
206         [
207             dx,
208             dθ,
209             (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
210             -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
211             / (h * L),
212         ]
213     )
214     return ds
215
216
217 def discretize(f, dt):
218     """Discretize continuous-time dynamics `f` via Runge-Kutta integration."""

```

```

219
220     def integrator(s, u, dt=dt):
221         k1 = dt * f(s, u)
222         k2 = dt * f(s + k1 / 2, u)
223         k3 = dt * f(s + k2 / 2, u)
224         k4 = dt * f(s + k3, u)
225         return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6
226
227     return integrator
228
229
230 # Define constants
231 n = 4 # state dimension
232 m = 1 # control dimension
233 s_goal = np.array([0, np.pi, 0, 0]) # desired upright pendulum state
234 s0 = np.array([0, 0, 0, 0]) # initial downright pendulum state
235 dt = 0.1 # discrete time resolution
236 T = 10.0 # total simulation time
237 P = 1e3 * np.eye(n) # terminal state cost matrix
238 Q = np.diag([1e-2, 1.0, 1e-3, 1e-3]) # state cost matrix
239 R = 1e-3 * np.eye(m) # control cost matrix
240 p = 1.0 # trust region parameter
241 u_max = 8.0 # control effort bound
242 eps = 5e-1 # convergence tolerance
243 max_iters = 100 # maximum number of SCP iterations
244 animate = False # flag for animation
245
246 # Initialize the discrete-time dynamics
247 fd = jax.jit(discretize(cartpole, dt))
248
249 # Solve the swing-up problem with SCP
250 t = np.arange(0.0, T + dt, dt)
251 N = t.size - 1
252 s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, p, eps, max_iters)
253
254 # Simulate open-loop control
255 for k in range(N):
256     s[k + 1] = fd(s[k], u[k])
257
258 # Plot state and control trajectories
259 fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
260 plt.subplots_adjust(wspace=0.45)
261 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
262 labels_u = (r"$u(t)$",)
263 for i in range(n):
264     ax[i].plot(t, s[:, i])
265     ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
266     ax[i].set_xlabel(r"$t$")
267     ax[i].set_ylabel(labels_s[i])
268 for i in range(m):
269     ax[n + i].plot(t[:-1], u[:, i])
270     ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
271     ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
272     ax[n + i].set_xlabel(r"$t$")
273     ax[n + i].set_ylabel(labels_u[i])
274 plt.savefig("cartpole_swingup_constrained.png", bbox_inches="tight")
275
276 # Plot cost history over SCP iterations
277 fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
278 ax.semilogy(J)
279 ax.set_xlabel(r"SCP iteration ${i}$")
280 ax.set_ylabel(r"SCP cost $\bar{J}(\bar{x}^{\{i\}}, \bar{u}^{\{i\}})$")
281 plt.savefig("cartpole_swingup_constrained_cost.png", bbox_inches="tight")
282 plt.show()
283
284 # Animate the solution
285 if animate:
286     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
287     ani.save("cartpole_swingup_constrained.mp4", writer="ffmpeg")
288     plt.show()

```